10. Critères 7 et 8 : rétention, compaction, réaffectation ; supervision, sécurité, remise en service

Guide de réalisation et dossier de preuves des phases H, I et J. TP fil rouge FROIDIS (MCCI, BC05, chapitre 2), Kafka 4.3.1 en KRaft à rôles séparés (contrôleurs 1 à 3, brokers 4 à 6). Version du 25/10/2026.

0. Ce que demandent les critères, et comment lire ce guide

N°	Critère de recette	Phases	Preuve attendue (sujet, p. 20)	Ce qu'elle doit montrer	Ici
7	Une démonstration de rétention, de compaction et de réaffectation de répliques	H, I	`kafka-dump-log.sh`, relectures (H) ; `--verify` et élection (I)	valeurs compactées, offsets inchangés ; leaders préférés rétablis	§2, §3
8	Un plan de supervision, de sécurité et de remise en service argumenté	J (+ L)	tableau de bord et procédures (J) ; arrêt et relance (L)	métriques expliquées ; « already formatted » au redémarrage	§4

Barème proposé : 2 points pour « compaction, réaffectation et leaders préférés », 2 points pour « supervision, sécurité, qualité des preuves ».

Mode d'emploi. Chaque étape donne l'objectif, les commandes, une sortie **réelle**, ce qu'il faut observer et l'explication à défendre. Les sorties viennent de deux sources :

- **réf. 22/10** : les preuves du groupe (`preuves/H1-segments.txt`, `H2-*`, `I1…I5`, `G4-*`, `L2`, `L3`), obtenues sur le poste de référence ;
- **poste 25/10** : un second poste, sur lequel le cluster a été reconstruit à partir du dépôt pour valider ce guide (`preuves/H1-segments-25-10.txt`, `H2-complement-segment-actif.txt`, `H3-retention.txt`, `I6-throttle-et-retour-arriere.txt`, `J-*.txt`).

Vos `producerId`, `TopicId`, horodatages et leaders seront différents : comparez la forme, les offsets et les états.

Conventions du sujet. *[hôte]* : terminal ouvert dans `projet-froid-kafka/`. Sous Git Bash, taper `export MSYS_NO_PATHCONV=1` dans chaque nouveau terminal ; si `docker exec -it` répond « the input device is not a TTY » (Git Bash hors de VS Code), préfixer la commande par `winpty`, ou ouvrir le terminal Kafka depuis PowerShell ou le terminal de VS Code. **Sans mention : terminal Kafka**, ouvert par `docker exec -it froid-kafka-1 bash` puis `cd /opt/kafka`, avec l'adresse `kafka-1:19092`. Le script `scripts/tableau-de-bord.sh` se lance depuis l'hôte, sous Git Bash (sous PowerShell : `& "$env:ProgramFiles\Git\bin\bash.exe" scripts/tableau-de-bord.sh`, car `bash` y désigne WSL). Ne tapez pas le `$`.

0.1 Avant de commencer : un cluster prêt

Le dépôt ne contient que la configuration : sur un poste qui vient de le cloner, il n'existe ni conteneur ni volume (constaté le 25/10). H et I exigent le cluster des phases C et D et les mesures de la phase E (≈ 10 min) :

```
[hôte] docker compose -f docker-compose-froid.yml up -d --wait   # formate, puis démarre les 6 nœuds
[hôte] docker exec -it froid-kafka-1 bash                        # puis : cd /opt/kafka
bin/kafka-metadata-quorum.sh --bootstrap-server kafka-1:19092 describe --status   # un LeaderId, 3 votants
# phase D : les 4 commandes « bin/kafka-topics.sh … --create » de preuves/D-topics.txt (sans le « $ »)
# phase E : ce producteur, puis COLLER D'UN SEUL COUP les 5 lignes du sujet (p. 11), sinon H1 montre 3 lots
bin/kafka-console-producer.sh --bootstrap-server kafka-1:19092 --topic mesures-brutes \
    --reader-property parse.key=true --reader-property key.separator=: \
    --command-property acks=all --command-property enable.idempotence=true
```

Incident réel du 25/10 (à présenter à l'oral). Pendant le premier démarrage, Docker Desktop a redémarré son moteur WSL (mémoire de l'hôte saturée : 0,4 Go libre sur 23,7 Go). Les 6 conteneurs sont passés en `Exited (255)`. Remise en service : `docker compose -f docker-compose-froid.yml start`, qui relance sans reformater (« All of the log directories are already formatted. ») ; 6 nœuds `healthy` 86 s après ce `start`. Comme `docker logs` restait ensuite figé pour les brokers, leurs conteneurs ont été recréés (`up -d --wait --force-recreate kafka-1 kafka-2 kafka-3`) : volumes conservés, données et identité intactes (`preuves/J-relance-apres-crash.txt`). Prévoir au moins 4 Go de mémoire libre.

1. Les notions à maîtriser

```
/var/lib/kafka/data/demo-retention-0      une partition = un répertoire = une suite de segments
┌── segment 0 (fermé) ──────┐┌── segment 5 (fermé) ──┐┌── segment 6 (ACTIF) ──┐
│ 00000000000000000000.log  ││ …0005.log  .index     ││ …0006.log  .index     │
│ offsets 0 → 4             ││ offset 5              ││ offsets 6 → …         │
└───────────────────────────┘└───────────────────────┘└───────────────────────┘
▲ début du journal (log start offset)                          fin (log end offset) ▲
delete  : supprime des segments ENTIERS dont le message le plus récent a dépassé retention.ms
compact : réécrit les segments FERMÉS en gardant au moins la dernière valeur de chaque clé
```

1. **Segment.** Une partition est découpée en segments, nommés d'après leur premier offset. Un seul est actif, celui où l'on écrit. Il est fermé quand il dépasse `segment.bytes` (1 Gio) ou `segment.ms` (7 jours), au moment où arrive le message suivant **dans cette partition**.
2. **Offset.** C'est une position dans la partition, jamais renumérotée : la rétention fait avancer le début du journal, la compaction crée des trous.
3. **Rétention `delete`.** Elle supprime des segments entiers, jugés sur leur message le plus récent, lors d'un contrôle périodique. Même le segment actif peut partir si tout son contenu a expiré : Kafka ouvre alors d'abord un segment vide. Sous un flux continu, cela n'arrive pas.
4. **Compaction `compact`.** Le nettoyeur garde **au moins** la dernière valeur de chaque clé dans les segments fermés. Il ne touche jamais le segment actif. Une valeur nulle (*tombstone*) annonce la suppression d'une clé.
5. **Réaffectation.** Elle change la **liste** des répliques d'une partition. Le premier broker de la liste est le **leader préféré** ; lui rendre le leadership est une **élection**, opération distincte.

Réglages en jeu, relevés le 25/10 sur le broker 4 (`kafka-configs.sh --describe --all`, `preuves/H1-segments-25-10.txt`) :

Réglage (topic / broker)	Rôle	Défaut	Démo
`segment.bytes` / `log.segment.bytes`	taille maximale d'un segment	1 073 741 824 (1 Gio) ; **minimum 1 048 576** en 4.x	—
`segment.ms` / `log.roll.hours`	âge maximal du segment actif	168 h	20 000 ms
`retention.ms` / `log.retention.hours`	âge au-delà duquel un segment est supprimé	168 h (`mesures-brutes`)	60 000 ms
`log.retention.check.interval.ms`	période du contrôle de rétention (non modifiable à chaud)	300 000 ms	—
`file.delete.delay.ms` / `log.segment.delete.delay.ms`	délai avant l'effacement des fichiers `.deleted`	60 000 ms	—
`min.cleanable.dirty.ratio` / `log.cleaner.min.cleanable.ratio`	part « sale » qui déclenche la compaction	0,5	0,01
`log.cleaner.backoff.ms`	pause du nettoyeur quand il n'a rien à faire	15 000 ms	—
`delete.retention.ms` / `log.cleaner.delete.retention.ms`	durée de vie d'un tombstone après compaction	86 400 000 (24 h)	—
`auto.leader.rebalance.enable`, `leader.imbalance.check.interval.seconds`	retour automatique aux leaders préférés	`true`, 300 s	—

2. Phase H : segments, rétention et compaction

H1. Lire le stockage sur disque (≈ 10 min)

Objectif : voir où et sous quelle forme les mesures de CAP-001 (phase E) sont écrites.

```
bin/kafka-configs.sh --bootstrap-server kafka-1:19092 --describe --all --entity-type brokers \
    --entity-name 4 | grep -E "^ *log\.(dirs|segment\.bytes|roll\.hours)="
ls /var/lib/kafka/data
ls -l /var/lib/kafka/data/mesures-brutes-1
bin/kafka-dump-log.sh --files /var/lib/kafka/data/mesures-brutes-1/00000000000000000000.log --print-data-log
```

Sortie (poste 25/10, juste après la phase E, `preuves/H1-segments-25-10.txt`) :

```
  log.dirs=/var/lib/kafka/data      log.roll.hours=168      log.segment.bytes=1073741824
-rw-r--r-- 1 appuser appuser 10485760 Oct 25 18:56 00000000000000000000.index
-rw-r--r-- 1 appuser appuser      367 Oct 25 18:57 00000000000000000000.log
-rw-r--r-- 1 appuser appuser 10485756 Oct 25 18:56 00000000000000000000.timeindex
baseOffset: 0 lastOffset: 2 count: 3 baseSequence: 0 lastSequence: 2 producerId: 0 producerEpoch: 0 partitionLeaderEpoch: 0
... size: 367 ...
| offset: 0 ... keySize: 7 valueSize: 86 sequence: 0 headerKeys: [] key: CAP-001 payload:
{"eventId":"E001",...,"temperatureC":4.2}
| offset: 1 ... keySize: 7 valueSize: 86 sequence: 1 headerKeys: [] key: CAP-001 payload:
{"eventId":"E003",...,"temperatureC":8.7}
| offset: 2 ... keySize: 7 valueSize: 86 sequence: 2 headerKeys: [] key: CAP-001 payload:
{"eventId":"E004",...,"temperatureC":9.0}
```

En réf. 22/10 (`preuves/H1-segments.txt`), le premier lot est le même, avec `producerId: 1000`, dans un `.log` de 1 121 o qui contient aussi les offsets 3 à 7 des phases E à G.

Élément	Comment le lire
`00000000000000000000.log`	Le segment porte le numéro de son **premier offset**. `.index` (offset → position) et `.timeindex` (horodatage → offset) sont **préalloués à 10 Mio** tant que le segment est actif, puis réduits à leur taille utile à sa fermeture (0 et 12 o dans `preuves/H3-retention.txt`, étape 3).
`baseOffset: 0 … lastOffset: 2`, `count: 3`	Un **lot** de 3 enregistrements, offsets 0 à 2.
`producerId`, `producerEpoch`, `baseSequence` … `lastSequence`	Identité du producteur et numéros de séquence : c'est l'**idempotence**.
`partitionLeaderEpoch`	Génération du leader qui a écrit le lot (+1 à chaque changement de leader).
`CreateTime`	Horodatage posé par le producteur, en ms : c'est lui que la rétention compare à `retention.ms`.
`position`, `size`	Emplacement et taille du lot dans le `.log` : 367 o pour 3 mesures de 93 o utiles (clé 7 o + valeur 86 o).

H-Q1. Pourquoi les trois mesures forment-elles un seul lot (`count: 3`) ? Le producteur regroupe les enregistrements **par partition** avant de les envoyer. Le producteur console attend jusqu'à 1 s pour remplir un lot (option `--timeout`, c'est-à-dire `linger.ms`, défaut 1000 ms d'après `--help`). Les cinq lignes collées d'un coup arrivent dans ce délai, et les trois de CAP-001, toutes destinées à la partition 1, partagent donc le même lot. Le lot est l'unité d'écriture, de réplication et de compression : en réf. 22/10, un lot d'une seule mesure occupe 163 o, contre 367 o pour trois. Tapées à plus d'une seconde d'intervalle, les mesures auraient formé trois lots.

H-Q2. À quoi servent `producerId` et `sequence` ? À l'idempotence. Le broker retient, pour chaque producteur et chaque partition, le dernier numéro de séquence écrit. Un lot renvoyé après la perte d'un acquittement porte le même couple (`producerId`, `sequence`) : il est écarté au lieu d'être écrit deux fois, et un trou dans la séquence est signalé comme une erreur d'ordre. Chaque lancement du producteur console reçoit un nouveau `producerId` (0 sur le cluster neuf du 25/10 ; 1000, 0, 1, 2 et 4 en réf. 22/10) et repart de la séquence 0. L'idempotence élimine donc les doublons **techniques** d'une session, pas un événement renvoyé par l'application (doc 05, E-Q3).

H-Q3. Quel lien entre offset, segment et politique de rétention ? Le segment est l'unité de stockage **et** de suppression. La rétention ne supprime pas les messages un par un : elle supprime un segment entier quand son message **le plus récent** a dépassé `retention.ms`. Le début du journal avance alors jusqu'au premier offset du segment suivant, et les offsets restants ne changent pas. Une mesure peut donc vivre plus de 7 jours. Au débit cible (≈ 16,7 ko/s par partition), un segment de 1 Gio se remplit en ≈ 18 h : on garde environ 7 jours + 18 h. Sur une partition peu active, le segment n'est fermé qu'au bout de `log.roll.hours`, ce qui conduit jusqu'à ≈ 14 jours. La démonstration H3 montre ce mécanisme en accéléré.

H2. Compacter `etat-capteurs` (≈ 15 min)

Objectif : montrer qu'un topic compacté garde la dernière valeur de chaque capteur sans renuméroter les offsets.

```
# Terminal Kafka 1 : réglages de DÉMONSTRATION (segment fermé après 20 s, nettoyage dès 1 % de données
# « sales »), puis producteur, à laisser ouvert
bin/kafka-configs.sh --bootstrap-server kafka-1:19092 --alter --entity-type topics \
    --entity-name etat-capteurs --add-config segment.ms=20000,min.cleanable.dirty.ratio=0.01
bin/kafka-console-producer.sh --bootstrap-server kafka-1:19092 --topic etat-capteurs \
    --reader-property parse.key=true --reader-property key.separator=:
```

```
# Lignes à coller dans le producteur ; puis, PLUS DE 20 s après la dernière : CAP-001:9.2
CAP-001:4.2
CAP-002:5.1
CAP-001:8.7
CAP-003:3.8
CAP-001:9.0
```

```
# Terminal Kafka 2 : lecture AVANT, puis environ 1 min après 9.2 ; ensuite, le segment compacté
bin/kafka-console-consumer.sh --bootstrap-server kafka-1:19092 --topic etat-capteurs --from-beginning \
    --formatter-property print.key=true --formatter-property print.partition=true \
    --formatter-property print.offset=true --timeout-ms 10000
ls -l /var/lib/kafka/data/etat-capteurs-1
bin/kafka-dump-log.sh --files /var/lib/kafka/data/etat-capteurs-1/00000000000000000000.log --print-data-log
```

```
# Terminal Kafka 1 : tombstone. Arrêter le producteur (Ctrl-C), le relancer avec null.marker, y taper
# CAP-002:NULL, relire dans le terminal 2, puis RETIRER les réglages de démonstration et vérifier
bin/kafka-console-producer.sh --bootstrap-server kafka-1:19092 --topic etat-capteurs \
    --reader-property parse.key=true --reader-property key.separator=: --reader-property null.marker=NULL
bin/kafka-configs.sh --bootstrap-server kafka-1:19092 --alter --entity-type topics \
    --entity-name etat-capteurs --delete-config segment.ms,min.cleanable.dirty.ratio
bin/kafka-configs.sh --bootstrap-server kafka-1:19092 --describe --entity-type topics --entity-name etat-capteurs
```

Sorties (réf. 22/10, `preuves/H2-compaction-avant.txt`, `H2-compaction-apres.txt`, `H2-tombstone-et-retrait.txt`) :

Avant	Une minute après `CAP-001:9.2`	Après le tombstone
`Partition:1 Offset:0 CAP-001 4.2`	`Partition:0 Offset:0 CAP-002 5.1`	`Partition:1 Offset:2 CAP-001 9.0`
`Partition:1 Offset:1 CAP-001 8.7`	`Partition:2 Offset:0 CAP-003 3.8`	`Partition:0 Offset:0 CAP-002 5.1`
`Partition:1 Offset:2 CAP-001 9.0`	`Partition:1 Offset:2 CAP-001 9.0`	`Partition:2 Offset:0 CAP-003 3.8`
`Partition:2 Offset:0 CAP-003 3.8`	`Partition:1 Offset:3 CAP-001 9.2`	`Partition:1 Offset:3 CAP-001 9.2`
`Partition:0 Offset:0 CAP-002 5.1`		`Partition:0 Offset:1 CAP-002 null`

```
-rw-r--r-- 1 appuser appuser       78 Oct 22 08:59 00000000000000000000.log    (segment fermé, compacté)
-rw-r--r-- 1 appuser appuser       78 Oct 22 09:00 00000000000000000003.log    (segment actif : 9.2)
| offset: 2 CreateTime: 1792659599150 keySize: 7 valueSize: 3 sequence: 2 headerKeys: [] key: CAP-001 payload: 9.0
Non-consecutive offsets in /var/lib/kafka/data/etat-capteurs-1/00000000000000000000.log
  -1 is followed by 2
Dynamic configs for topic etat-capteurs are:      cleanup.policy=compact ...      min.insync.replicas=2 ...
```

À observer. Le segment `…0000.log` ne contient plus que l'offset 2 (« Non-consecutive offsets »), le tombstone est relu comme `null`, et une fois les réglages retirés il ne reste que `cleanup.policy=compact` et `min.insync.replicas=2`.

> **Pièges.** (1) Gardez la clé **CAP-001** pour 9.2 : un segment ne se ferme qu'à l'arrivée d'un message dans **sa** partition (CAP-001 → partition 1, CAP-002 → 0, CAP-003 → 2). (2) Attendez **plus** de 20 s : sinon 9.2 entre dans le même segment et rien n'est compactable. (3) Le nettoyeur est asynchrone (pause de 15 s) : si 4.2 est encore là, attendez 30 s et relisez. (4) `ls` dans `froid-kafka-1` montre la réplique du broker 4 ; chaque réplique se compacte de son côté, alors que la lecture passe par le leader. (5) Le `ERROR … TimeoutException` qui termine le consommateur est la fin normale de `--timeout-ms`. (6) N'oubliez pas le retrait des réglages : la liste de clôture exige « aucun réglage de démonstration restant ». (7) Ne rejouez pas H2 sur `etat-capteurs`, dont les offsets continuent : prenez un topic jetable créé comme lui (`demo-compaction`, supprimé à la fin), comme le 25/10.

H2-Q1. Quelles valeurs ont disparu ? Les offsets ont-ils été renumérotés ? Les valeurs 4.2 (offset 0) et 8.7 (offset 1) de CAP-001 ont disparu. Les offsets restants gardent leur numéro (2 et 3) : la compaction crée des trous, elle ne renumérote jamais. Les positions déjà validées par les consommateurs restent donc valables.

H2-Q2. Pourquoi 9.0 reste-t-elle, alors que 9.2 est plus récente ? Le nettoyeur ne travaille que sur les segments **fermés** et ignore le segment actif, qui contient 9.2. Dans le segment fermé (offsets 0 à 2), la valeur la plus récente de CAP-001 est 9.0 : elle est gardée. 9.0 ne disparaîtra qu'après la fermeture du segment de 9.2, au passage suivant du nettoyeur. La compaction garantit « **au moins** la dernière valeur par clé », pas « exactement une », et le moment du nettoyage n'est pas garanti. Vérifié sur le poste 25/10 (topic `demo-compaction`, `preuves/H2-complement-segment-actif.txt`) : `CAP-001:9.5`, envoyée plus de 20 s après 9.2, ferme le segment de 9.2 ; une minute plus tard, 9.0 a disparu et il reste `Offset:3 CAP-001 9.2` et `Offset:4 CAP-001 9.5`.

H2-Q3. Pourquoi `cleanup.policy=compact` ne remplace-t-il pas la conservation de tout l'historique ? La compaction détruit les valeurs intermédiaires. On ne saurait plus que CAP-001 est passé par 8.7 puis 9.0, ni combien de temps il est resté au-dessus de 8 °C. La règle « plus de 8 °C pendant 2 minutes », le rejeu d'un incident et l'analyse différée deviennent impossibles. Même les tombstones finissent par disparaître (`delete.retention.ms`, 24 h). D'où deux topics : `mesures-brutes` (`delete`, 7 jours) pour l'historique, et `etat-capteurs` (`compact`) pour le dernier état.

H3. Démontrer la rétention (≈ 10 min, en complément du sujet)

Pourquoi. Le critère 7 demande une démonstration de rétention. H1 montre les segments, mais rien n'est supprimé avant 7 jours. On réduit donc la rétention **sur un topic de démonstration**, jamais sur `mesures-brutes`. Le contrôle de rétention ne passe que toutes les 5 min : on vise donc un passage précis.

```
# Terminal Kafka 1 : topic de démonstration (1 partition, répliques 4,5,6 : leader 4, disque de froid-kafka-1)
bin/kafka-topics.sh --bootstrap-server kafka-1:19092 --create --topic demo-retention \
    --replica-assignment 4:5:6 --config retention.ms=60000 --config segment.ms=20000 \
    --config min.insync.replicas=2
# Heures des contrôles du broker 4 : 30 s après « Starting log cleanup », puis toutes les 5 min
grep -h "Starting log cleanup" /opt/kafka/logs/server.log* | tail -1   # 25/10 : 18:54:44 → 18:55:14, 19:00:14, 19:05:14…
date -u +%T                                                            # heure UTC, comme le journal
bin/kafka-console-producer.sh --bootstrap-server kafka-1:19092 --topic demo-retention \
    --reader-property parse.key=true --reader-property key.separator=:
# Dans le producteur : les 5 lignes de H2 environ 1 min 40 avant le contrôle visé,
# CAP-001:9.2 environ 35 s avant ; puis, après la suppression, CAP-002:5.3
```

```
# Terminal Kafka 2 : relevés toutes les 30 s autour du contrôle, puis retrait du topic
bin/kafka-get-offsets.sh --bootstrap-server kafka-1:19092 --topic demo-retention --time earliest
bin/kafka-get-offsets.sh --bootstrap-server kafka-1:19092 --topic demo-retention --time latest
```

```
ls -l /var/lib/kafka/data/demo-retention-0
grep demo-retention-0 /opt/kafka/logs/server.log | grep -E "Deleting segment|log start offset"
bin/kafka-console-consumer.sh --bootstrap-server kafka-1:19092 --topic demo-retention --from-beginning \
    --formatter-property print.key=true --formatter-property print.offset=true --timeout-ms 8000
bin/kafka-topics.sh --bootstrap-server kafka-1:19092 --delete --topic demo-retention
# [hôte] répliques suiveuses : docker exec froid-kafka-2 ls /var/lib/kafka/data/demo-retention-0
```

Sortie (poste 25/10, `preuves/H3-retention.txt`) :

Heure (UTC)	Événement	Début / fin du journal	Fichiers
19:03:43	5 valeurs écrites, offsets 0 à 4 (un lot de 146 o)	0 / 5	broker 4 : `…0000.log` actif
19:04:54	`CAP-001:9.2` écrite à l'offset 5, 71 s après le lot ; segment 5 ouvert à 19:04:55	0 / 6	broker 4 : `…0000.log` fermé, `…0005.log` actif
19:05:14	contrôle de rétention : segment 0 supprimé (dernier message âgé de 91 s)	**5** / 6	broker 4 : `…0000.log.deleted`
19:06:14	effacement physique, 60 s plus tard	5 / 6	broker 4 : `…0005.*` seulement
19:07:34	`CAP-002:5.3` écrite à l'offset **6**, dans un nouveau segment 6	5 / 7 (relecture : offsets 5 et 6)	brokers 5 et 6 : `…0005.*`, `…0006.*`

```
INFO [UnifiedLog partition=demo-retention-0, dir=/var/lib/kafka/data] Incremented log start offset to 5 due to segment
deletion
INFO [UnifiedLog partition=demo-retention-0, dir=/var/lib/kafka/data] Deleting segment LogSegment(baseOffset=0, size=146,
lastModifiedTime=1792955023755, largestRecordTimestamp=1792955023537) due to log retention time 60000ms breach based on the
largest record timestamp in the segment
$ bin/kafka-console-consumer.sh ... --topic demo-retention --from-beginning ...
Offset:5    CAP-001 9.2
```

À observer et à expliquer.

- La rétention juge un segment sur son message **le plus récent** (« *based on the largest record timestamp in the segment* »). Le segment 0 (dernier message à 19:03:43) est supprimé ; le segment 5 (9.2, écrite 20 s avant le contrôle) est conservé.
- La suppression attend le contrôle périodique (`log.retention.check.interval.ms`, 5 min) : en production, un segment expiré peut survivre quelques minutes. Si vous visez mal, deux cas restent corrects : rien n'a encore expiré (attendre le contrôle suivant), ou 9.2 a expiré elle aussi et le journal se vide (début = fin).
- Les fichiers sont d'abord renommés en `.deleted`, puis effacés 60 s plus tard (`log.segment.delete.delay.ms`), le temps que les lectures en cours se terminent.
- Le début du journal passe de 0 à 5 et rien n'est renuméroté : la valeur suivante prend l'offset 6. Les répliques des brokers 5 et 6 commencent elles aussi à 5.
- Conséquence d'exploitation : un groupe dont l'offset validé est antérieur au début du journal repart selon `auto.offset.reset`. Avec `latest` (défaut), il saute à la fin et ignore **tout** ce qui reste dans le topic ; avec `earliest`, il ne perd que les segments supprimés. Les offsets d'un groupe resté vide expirent eux-mêmes après `offsets.retention.minutes` (7 jours par défaut), même sur un topic gardé 30 jours. D'où `earliest` pour les groupes d'analyse et d'alerte, et une alerte sur le retard bien avant ces échéances (§4.1).

Un même flux, deux politiques.

	`compact` (H2)	`delete` (H3)
Ce qui décide	la clé : une valeur plus récente existe	l'âge : le message le plus récent du segment dépasse `retention.ms`
Ce qui est supprimé	des enregistrements, dans les segments fermés	des segments entiers
Résultat pour CAP-001	4.2 et 8.7 supprimées ; 9.0 et 9.2 gardées	4.2, 8.7 et 9.0 supprimées ; 9.2 gardée
Offsets	des trous, jamais de renumérotation	le début avance (0 → 5), jamais de renumérotation
Topics FROIDIS	`etat-capteurs`	`mesures-brutes` (7 j), `mesures-agregees` et `alertes-temperature` (30 j)

Point de contrôle H. Un segment lu avec `kafka-dump-log.sh` ; les offsets 0 et 1 de CAP-001 disparus, les autres inchangés ; un tombstone visible ; les réglages de démonstration retirés. En complément : le début du journal passé de 0 à 5 sans renumérotation, puis le topic de démonstration supprimé.

3. Phase I : réaffecter des répliques et rétablir les leaders préférés (≈ 35 min)

Pourquoi en production. On réaffecte pour ajouter ou retirer un broker, rééquilibrer les disques ou augmenter le facteur de réplication : `kafka-topics.sh --alter` ne sait qu'ajouter des partitions, un changement de RF passe forcément par une réaffectation. Au TP, on travaille sur `tp-migration`, jamais sur un topic métier.

```
# I1. Topic à une seule réplique
bin/kafka-topics.sh --bootstrap-server kafka-1:19092 --create --topic tp-migration --partitions 3 --replication-factor 1
bin/kafka-topics.sh --bootstrap-server kafka-1:19092 --describe --topic tp-migration
# I2. Demander un plan, puis GARDER VOTRE plan de retour arrière (la ligne JSON qui suit
#     « Current partition replica assignment »)
echo '{"topics": [{"topic": "tp-migration"}], "version": 1}' > /tmp/topics-migration.json
bin/kafka-reassign-partitions.sh --bootstrap-server kafka-1:19092 \
    --topics-to-move-json-file /tmp/topics-migration.json --broker-list 4,5,6 --generate
cat > /tmp/retour-arriere.json <<'FIN'
<coller ici, à la place de cette ligne, le JSON qui suit « Current partition replica assignment »>
FIN
# I3. Écrire le plan DANS le conteneur : coller le bloc d'un coup, de « cat » à « FIN »
cat > /tmp/plan-migration.json <<'FIN'
{"version": 1, "partitions": [
  {"topic": "tp-migration", "partition": 0, "replicas": [4, 5, 6]},
  {"topic": "tp-migration", "partition": 1, "replicas": [5, 6, 4]},
  {"topic": "tp-migration", "partition": 2, "replicas": [6, 4, 5]}]}
FIN
# I4. Exécuter, vérifier (relancer --verify jusqu'à « completed »), décrire ; enchaîner aussitôt avec I5
bin/kafka-reassign-partitions.sh --bootstrap-server kafka-1:19092 --reassignment-json-file /tmp/plan-migration.json --execute
bin/kafka-reassign-partitions.sh --bootstrap-server kafka-1:19092 --reassignment-json-file /tmp/plan-migration.json --verify
bin/kafka-topics.sh --bootstrap-server kafka-1:19092 --describe --topic tp-migration
# I5. Élection PREFERRED limitée à ce topic (--all-topic-partitions toucherait tout le cluster)
cat > /tmp/election-migration.json <<'FIN'
{"partitions": [
  {"topic": "tp-migration", "partition": 0},
  {"topic": "tp-migration", "partition": 1},
  {"topic": "tp-migration", "partition": 2}]}
FIN
bin/kafka-leader-election.sh --bootstrap-server kafka-1:19092 --election-type PREFERRED \
    --path-to-json-file /tmp/election-migration.json
bin/kafka-topics.sh --bootstrap-server kafka-1:19092 --describe --topic tp-migration
# puis relancer une seconde fois la même élection
```

Variante, depuis l'hôte, avec les fichiers du dossier `json/` (LF dans le dépôt, sans BOM ; une extraction Git sous Windows peut les passer en CRLF, ce que le JSON tolère) :

```
[hôte, Git Bash]   docker exec -i froid-kafka-1 bash -c 'cat > /tmp/plan-migration.json' < json/plan-migration.json
[hôte, PowerShell] Get-Content -Raw json/plan-migration.json | docker exec -i froid-kafka-1 bash -c 'cat > /tmp/plan-migration.json'
```

Sorties (réf. 22/10, `preuves/I1-I2-generate.txt`, `I3-I4-reaffectation.txt`, `I5-election-preferee.txt`) :

Étape	Partition 0	Partition 1	Partition 2	Message clé
Création (RF 1)	Leader 5, Replicas 5	Leader 6, Replicas 6	Leader 4, Replicas 4	`Created topic tp-migration.`
`--execute` puis `--verify` (RF 3)	Leader **5**, Replicas 4,5,6	Leader **6**, Replicas 5,6,4	Leader **4**, Replicas 6,4,5	`Reassignment of partition tp-migration-0 is completed.` (×3), puis `Clearing broker-level throttles on brokers 4,5,6`
Élection PREFERRED	Leader **4**	Leader **5**	Leader **6**	`Successfully completed leader election (PREFERRED) for partitions tp-migration-2, tp-migration-1, tp-migration-0`
Seconde élection	inchangé	inchangé	inchangé	`Valid replica already elected for partitions tp-migration-2, tp-migration-1, tp-migration-0`

Après `--verify`, l'ISR vaut 4,5,6 sur les trois partitions. Le fichier `json/retour-arriere-migration.json` est le plan de retour du poste de référence (répliques 5, 6, 4) : sur un autre cluster, utilisez le vôtre (`/tmp/retour-arriere.json`).

I-Q1. Pourquoi garder le plan initial ? À quoi servent `--generate`, `--execute` et `--verify` ? Le plan initial est le **retour arrière** : une fois la réaffectation terminée, relancé avec `--execute`, il remet chaque partition sur ses répliques d'origine, par exemple après un mauvais placement (joué le 25/10, ci-dessous). Pendant la copie, en cas de saturation du réseau ou du disque, un nouveau `--execute` est refusé (sauf avec `--additional`) : on baisse le quota (`--execute --additional --throttle <octets/s>`, même plan) ou on annule (`--cancel`), ce qui rend à chaque partition ses répliques d'avant. `--generate` ne modifie rien et conserve le facteur de réplication : en I2, il propose de déplacer l'unique réplique de chacune des trois partitions (5 → 4, 6 → 5, 4 → 6), toujours en RF 1 ; passer à RF 3 exige donc un plan écrit à la main. `--execute` lance la copie, qui est asynchrone : les nouvelles répliques rattrapent le leader et entrent dans l'ISR, puis les répliques absentes du plan sont supprimées. `--verify` contrôle la fin de chaque partition et **retire les quotas** de réplication posés par `--throttle` : il faut le relancer jusqu'à « completed ».

I-Q2. Pourquoi le premier broker de `Replicas` n'est-il pas leader dès la fin de la réaffectation ? La réaffectation change la liste des répliques, pas le leader. Tant que le leader actuel figure dans la nouvelle liste et dans l'ISR, Kafka ne le déplace pas, ce qui évite d'interrompre inutilement les clients.

I-Q3. Que répond Kafka à une seconde élection ? Quel réglage l'aurait fait seul ? « *Valid replica already elected* » : le leader préféré est déjà leader, il n'y a rien à faire, et la commande réussit (code retour 0). Avec `auto.leader.rebalance.enable=true` (défaut), le contrôleur actif vérifie toutes les `leader.imbalance.check.interval.seconds` (300 s) et rend le leadership à chaque leader préféré présent dans l'ISR. C'est ce qui s'est passé en G4, avec un retour à T+293 s. Le seuil `leader.imbalance.per.broker.percentage` des anciennes versions de Kafka n'existe plus en 4.3.1 : il est absent du `--describe --all` du 25/10.

En production : quotas et retour arrière (poste 25/10, topic `demo-migration` de 20 000 mesures, `preuves/I6-throttle-et-retour-arriere.txt`) :

```
bin/kafka-reassign-partitions.sh ... --reassignment-json-file /tmp/plan-demo.json --execute --throttle 1048576
Warning: You must run --verify periodically, until the reassignment completes, to ensure the throttle is removed.
The inter-broker throttle limit was set to 1048576 B/s
  follower.replication.throttled.rate=1048576   leader.replication.throttled.rate=1048576       (broker 4, pendant la copie)
  follower.replication.throttled.replicas=0:4,0:5,1:5,1:6,2:4,2:6   leader.replication.throttled.replicas=0:6,1:4,2:5
(topic)
Clearing broker-level throttles on brokers 4,5,6                                              (--verify, copie terminée)
Clearing topic-level throttles on topic demo-migration
```

`--throttle` plafonne le débit de copie entre brokers (ici 1 Mio/s), pour ne pas saturer le réseau ni ralentir les producteurs et les consommateurs. Kafka pose le quota sur les brokers et la liste des répliques concernées sur le topic. Le journal de métadonnées KRaft montre que les nouvelles répliques sont entrées dans l'ISR en 3 s, puis qu'à 19:28:31 seul le leader a changé : c'est le rééquilibrage automatique (I-Q3), et l'élection lancée ensuite a répondu « Valid replica already elected ». Le retour arrière, avec le bloc « Current » sauvegardé, a ramené le topic à RF 1 sur ses répliques d'origine, avec les mêmes offsets de fin (6600, 6120, 7280) : aucune donnée perdue.

> **Pièges.** (1) Écrivez le JSON dans le conteneur : un fichier enregistré par un éditeur Windows peut commencer par un BOM, et le plan devient illisible. (2) Vérifiez le nom du topic et les numéros de brokers ; vos leaders à la création peuvent différer, le plan et le résultat final restent les mêmes. (3) Enchaînez I4 et I5 : si le contrôle automatique (≤ 300 s, 1 min le 25/10) passe entre-temps, la première élection répond déjà « Valid replica already elected ». Ce n'est pas une erreur : montrez le `--describe` d'avant et expliquez I-Q3. (4) Tant que `tp-migration` n'a qu'une réplique (I1 à I3), `kafka-topics.sh --under-min-isr-partitions`, et donc le tableau de bord, le classent « sous le min ISR », car l'outil compare l'ISR à la valeur brute de `min.insync.replicas` (2). Le broker, lui, applique un minimum effectif de min(2, RF) = 1 : il accepte les écritures `acks=all` (vérifié le 25/10 sur un topic de RF 1 : écriture acceptée, `preuves/I7-rf1-min-isr-effectif.txt`), et la métrique `UnderMinIsrPartitionCount` reste à 0. L'alerte disparaît au passage à RF 3. (5) Pour rejouer : `bin/kafka-topics.sh --bootstrap-server kafka-1:19092 --delete --topic tp-migration`, puis recommencer (topic de démonstration).

Point de contrôle I. `ReplicationFactor: 3` pour `tp-migration`, puis des leaders égaux au premier broker de chaque liste ; une seconde élection répond « Valid replica already elected ».

4. Phase J : plan de supervision, de sécurité et de remise en service

4.1 Tableau de bord de supervision et plan d'intervention

En production, les métriques JMX des brokers et des contrôleurs sont exportées (JMX Exporter) vers Prometheus et affichées dans Grafana. Le sujet ne demande pas de les installer. Chaque indicateur principal a un équivalent en ligne de commande, et ces équivalents sont regroupés dans **`scripts/tableau-de-bord.sh`**, à lancer depuis l'hôte : lecture seule, un verdict OK / AVERTISSEMENT / CRITIQUE par indicateur, code retour 0, 1 ou 2. Les compléments de la dernière ligne ne se lisent que par JMX, non exposé au laboratoire.

Indicateur et métrique JMX	Sain	Alerte	Ce que cela signifie	Plan d'intervention (première action)
Sous-réplication : `kafka.server:type=ReplicaManager,name=UnderReplicatedPartitions` (chaque broker)	0	> 0 : avertissement (attendu pendant une maintenance) ; > 0 pendant plus de 5 min hors maintenance : critique	Une réplique a quitté l'ISR : broker arrêté, disque ou réseau lent (`replica.lag.time.max.ms` = 30 s). 65 partitions en G2.	`--describe --under-replicated-partitions` : quel broker manque dans `Isr` ? Lire son `server.log`. Suspendre toute maintenance.
Sous le min ISR : `…,name=UnderMinIsrPartitionCount` (`AtMinIsrPartitionCount` en pré-alerte)	0	> 0 : **critique** ; au min ISR : avertissement	Les écritures `acks=all` sont refusées (G3) : les passerelles des capteurs doivent tamponner. Exception : un topic de RF 1 (voir §3, piège 4).	Redémarrer le broker manquant. Ne jamais baisser `min.insync.replicas` pour « faire passer » les écritures.

Indicateur et métrique JMX	Sain	Alerte	Ce que cela signifie	Plan d'intervention (première action)
Partitions hors ligne : `kafka.controller:type=KafkaController,name=OfflinePartitionsCount` (publiée par chaque contrôleur : prendre le maximum ; topics non internes)	0	> 0 : **critique**	Aucun leader : ni lecture ni écriture sur ces partitions.	`--unavailable-partitions` (colonnes `Isr`, `Elr`), puis relancer le dernier leader ou un broker de `Elr` : eux seuls peuvent redevenir leader sans perte. Élection impure (`--election-type UNCLEAN`) seulement sur décision écrite.
Contrôleur actif : `…KafkaController,name=ActiveControllerCount` (somme sur les 3) et `kafka.server:type=raft-metrics` (`current-leader`)	somme = 1 ; même `current-leader` partout ; `Lag` ≈ 0 (`describe --replication`)	0 ou > 1 : **critique** ; un contrôleur arrêté : avertissement	Sans leader du quorum, plus d'élection, de changement d'ISR ni de création de topic.	`kafka-metadata-quorum.sh describe --status` et `--replication`, puis relancer le contrôleur arrêté (2 sur 3 suffisent).
Retard des consommateurs : `records-lag-max` (client), LAG par groupe	stable, proche de 0	> 1 000 messages pour un groupe (seuil du script), hausse continue, ou retard qui approche la rétention ou `offsets.retention.minutes`	Alertes tardives ; au-delà de la rétention, des mesures sont perdues pour ce groupe (H3).	`kafka-consumer-groups.sh --describe --group …`, puis relancer ou ajouter des consommateurs (6 au plus par groupe).
Disque : occupation de `log.dirs` (`df`), `kafka.log:type=Log,name=Size`, `kafka.log:type=LogManager,name=OfflineLogDirectoryCount`	< 70 % occupé (visé) ; 0 répertoire hors ligne	≥ 80 % : avertissement ; ≥ 90 % ou répertoire hors ligne : critique	Un disque plein ou en erreur arrête le broker.	`kafka-log-dirs.sh --describe` et `df`. Vérifier la rétention, ajouter du disque ; en cas de panne : §4.4.
Réglages de démonstration (liste de clôture de la phase L)	aucun	`segment.ms` ou `min.cleanable.dirty.ratio` sur `etat-capteurs`, ou topic `demo-*` : avertissement	Une démonstration oubliée change la rétention ou la compaction d'un topic.	Retirer le réglage (`--delete-config`), supprimer le topic de démonstration.
Journaux : `ERROR` / `FATAL` dans `server.log`	aucune nouvelle	toute nouvelle ligne : avertissement	Point de départ du diagnostic des alertes ci-dessus.	`docker logs --since 15m <conteneur>`, filtré sur ERROR et FATAL, puis classer : bruit de démarrage connu, bascule du quorum, ou vraie panne.
Compléments (JMX seulement) : `IsrShrinksPerSec` / `IsrExpandsPerSec`, `RequestHandlerAvgIdlePercent` (fraction de 0 à 1), `TotalTimeMs` (Produce), `PreferredReplicaImbalanceCount`	stables ; inactivité > 0,3	rafales ; inactivité < 0,2	ISR instable, broker saturé, leaders déséquilibrés.	Élection PREFERRED (§3), revoir la capacité (doc 02).

Sortie réelle (poste 25/10, après la remise en service, `preuves/J-tableau-de-bord.txt`, relevé 2) :

```
[hôte] bash scripts/tableau-de-bord.sh
=== Tableau de bord FROIDIS : 25/10/2026 23:05:47 (journaux : 15m) ===
[OK           ] Contrôleur actif (quorum)         leader 1, 3 votants en service, retard max 0
[OK           ] Brokers actifs                    3 / 3 (ids 4, 5, 6)
[OK           ] Sous-réplication (URP)            0 partition
[OK           ] Sous le min ISR                   0 partition (et 0 au min ISR)
[OK           ] Partitions hors ligne             0 partition
[OK           ] Leaders préférés                  toutes les partitions métier
[OK           ] Retard des consommateurs (LAG)    aucun groupe avec offsets validés
[OK           ] Disque froid-kafka-1              2 % occupé, données Kafka 1.7M
   ... (froid-kafka-2 et froid-kafka-3 : 2 %)
[OK           ] Répertoires de journaux           3 en ligne, 0 hors ligne
[OK           ] Réglages de démonstration         aucun sur etat-capteurs, aucun topic demo-*
[AVERTISSEMENT] Journal froid-controller-1        101 ERROR/FATAL : lire docker logs --since 15m froid-controller-1
[OK           ] Journal froid-controller-2        0 ERROR/FATAL
[AVERTISSEMENT] Journal froid-controller-3        29 ERROR/FATAL : lire docker logs --since 15m froid-controller-3
   ... (froid-kafka-1 à froid-kafka-3 : 0)
=== Bilan : avertissement(s) à examiner ===
```

Application du plan : les 130 erreurs, une fois lues, sont toutes des `NotControllerException` sur les heartbeats des brokers, émises entre 20:07 et 20:08 UTC pendant des bascules du leader du quorum. L'hôte Windows était saturé (0,5 Go de RAM libre), et aucune partition n'a été touchée. Conclusion : cause environnementale, consignée, sans action Kafka. Un relevé fait juste après le

démarrage a montré de la même façon le bruit connu du démarrage (« Unexpected error handling request CONTROLLER_REGISTRATION », signalé par le sujet ; relevé 1). Le tableau ne donne qu'un instantané : pour une tendance (un LAG qui monte, une sous-réplication qui dure plus de 5 min), il faut des relevés répétés, ce que fait Prometheus.

> **Incident réel du 25/10.** Sous la pression mémoire de l'hôte, le leader du quorum a changé à 19:28:41, puis plusieurs fois entre 19:39:51 et 19:40:06 (contrôleur 3 → 1 → 3 → 2) et entre 20:07 et 20:08, à chaque fois en quelques secondes. Seule trace : des `NotControllerException` sur les heartbeats des brokers, aussitôt rejoués, et aucune partition n'a perdu son leader. En production, des bascules répétées (changements de `current-leader`) sont un avertissement : il faut vérifier le CPU, la mémoire et les pauses GC des contrôleurs, puis le réseau.

4.2 Maintenance progressive d'un broker (procédure jouée le 25/10)

Elle s'appuie sur l'arrêt contrôlé observé en G2 : `docker stop` envoie SIGTERM et, avec `controlled.shutdown.enable=true`, le broker cède ses leaderships aux autres répliques de l'ISR avant de s'arrêter. Preuve : `preuves/J-maintenance-progressive.txt`.

1. **Pré-contrôles.** *[hôte]* `bash scripts/tableau-de-bord.sh` ne doit signaler aucun CRITIQUE, et chaque AVERTISSEMENT doit être lu, expliqué et consigné avant de commencer (le 25/10 : 4 `NotControllerException` transitoires sur `froid-controller-3`, dues à la bascule du quorum de 19:28:41). Aucune réaffectation ne doit être en cours : `bin/kafka-reassign-partitions.sh --bootstrap-server kafka-1:19092 --list`. On ne commence jamais une maintenance sur un cluster dégradé.
2. **Un seul broker à la fois**, donc une seule zone. Deux brokers arrêtés ensemble font tomber l'ISR à 1, sous le minimum de 2, et bloquent les écritures (G3).
3. **Arrêt contrôlé :** *[hôte]* `docker stop -t 120 froid-kafka-3` (en production : `systemctl stop kafka`, jamais `kill -9`). Vérifier que l'ISR est passé à 2 et que les écritures `acks=all` passent. Le 25/10 (`docker stop` sans `-t`) : arrêt en 7 s, dont 2,4 s d'arrêt contrôlé (« Beginning controlled shutdown », puis « The controller has asked us to exit controlled shutdown ») ; les partitions 0 et 4 de `mesures-brutes` passent du leader 6 aux leaders 4 et 5, l'ISR vaut {4,5}, 65 partitions sont sous-répliquées et une écriture `acks=all` est acceptée (code retour 0).
4. **Intervention** : mise à jour, changement de configuration, travaux sur le système.
5. **Relance :** *[hôte]* `docker start froid-kafka-3`, puis attendre la fin de la sous-réplication par une boucle plutôt qu'une durée fixe. Le 25/10 : encore 65 partitions sous-répliquées à T+51 s, 0 à T+75 s (12 s en réf. 22/10, la durée dépend du poste).

   ```
   while bin/kafka-topics.sh --bootstrap-server kafka-1:19092 --describe --under-replicated-partitions | grep -q Partition:;
   do sleep 5; done; echo "0 partition sous-répliquée"
   ```

6. **Leaders préférés :** attendre le contrôle automatique (300 s au plus) ou lancer `bin/kafka-leader-election.sh --bootstrap-server kafka-1:19092 --election-type PREFERRED --all-topic-partitions`. Ici, toucher tout le cluster est voulu : le 25/10, l'élection a rendu 21 partitions à leur leader préféré, dont `mesures-brutes-0` et `mesures-brutes-4`.
7. **Contrôle final :** tableau de bord sans CRITIQUE ni avertissement inexpliqué, retard des consommateurs normal. Consigner l'intervention dans `Command.md`, puis passer au broker suivant.
8. **Contrôleurs :** même principe, un seul à la fois, en vérifiant entre deux arrêts que le quorum a un leader et que le `Lag` des votants revient à 0. La majorité est de 2 sur 3 : on n'en arrête jamais deux.

4.3 Sécurité : choix justifiés et cible de production

Choix	Argument
`unclean.leader.election.enable=false`	Une réplique hors ISR peut être en retard. Élue leader, elle ferait **disparaître** des mesures déjà confirmées, et les consommateurs verraient les offsets reculer. On préfère une partition momentanément indisponible à une perte silencieuse. ELR (*Eligible Leader Replicas*, KIP-966 ; doc 05, G-3) va plus loin : une réplique sortie de l'ISR alors qu'il était déjà au min ISR reste éligible, puisque plus aucune écriture n'a pu être confirmée sans elle ; si le dernier membre de l'ISR tombe, elle peut devenir leader sans perte et sans élection impure.
`broker.rack` = A, B, C	Kafka place les 3 répliques dans 3 zones : la perte d'une zone (courant, réseau, salle) ne coûte qu'une réplique, et l'ISR reste à 2, au niveau du min ISR. Les consommateurs peuvent lire la réplique de leur zone (`client.rack`) si les brokers ont `replica.selector.class=org.apache.kafka.common.replica.RackAwareReplicaSelector` (KIP-392) ; sans ce réglage, cas du laboratoire, toute lecture passe par le leader.
Sauvegardes adaptées	La réplication protège d'une panne de matériel isolée, pas d'une erreur logique (§4.5). MirrorMaker 2 vers un cluster distant protège d'un sinistre de site, mais il recopie lui aussi les données erronées et, par défaut, les changements de configuration des topics : ce n'est pas une sauvegarde. À sauvegarder à chaque changement : `.env` (`CLUSTER_ID`, UUID), `config/*.properties`, `docker-compose-froid.yml`, la définition des topics et des ACL ; et les données à garder au-delà de la rétention, par export vers un stockage objet versionné.
Isolation des réseaux	Le port 9093 des contrôleurs n'est joignable que par les brokers et les autres contrôleurs. Le trafic entre brokers passe sur un réseau dédié, les clients ne voient que le listener client, les capteurs passent par des passerelles et l'administration par un bastion. Un pare-feu filtre les ports selon le rôle.

Choix	Argument
TLS et SASL en production	PLAINTEXT transporte les mesures et les identifiants **en clair** et accepte n'importe quel client. Cible : TLS sur tous les listeners, authentification SASL/SCRAM-SHA-512 (ou OAUTHBEARER) des applications et mTLS des brokers et contrôleurs, autorisations par ACL, secrets hors du dépôt et renouvelés.

Autorisations minimales de la cible, avec `allow.everyone.if.no.acl.found=false` : tout ce qui n'est pas autorisé est refusé.

Application (principal)	Ressources	Opérations
`User:passerelle-capteurs`	topic `mesures-brutes`	Write, Describe
`User:flink-froid`	topic `mesures-brutes` ; topics `mesures-agregees`, `alertes-temperature`, `etat-capteurs` ; groupe `flink-froid` ; `TransactionalId` de préfixe `flink-froid-` (motif `prefixed`)	Read ; Write ; Read ; Write et Describe (sorties *exactly-once*)
`User:tableau-de-bord`	topics `mesures-brutes`, `etat-capteurs`, `alertes-temperature` ; groupe `supervision-froid`	Read, Describe ; Read
`User:analyse-qualite`	topic `mesures-brutes` ; groupe `analyse-qualite`	Read, Describe ; Read
`User:service-alertes`	topic `alertes-temperature` ; groupe `service-alertes`	Read, Describe ; Read
`User:admin-froid`	cluster	super-utilisateur (`super.users`) : il échappe à toutes les ACL, restrictions d'hôte comprises ; l'accès « depuis le bastion seulement » s'impose donc par le réseau.

```
# Cible (extrait de broker-N.properties, à tester en préproduction ; non activé au laboratoire)
listeners=CLIENT://kafka-1:9094,INTERNE://kafka-1:9095
listener.security.protocol.map=CLIENT:SASL_SSL,INTERNE:SSL,CONTROLLER:SSL
inter.broker.listener.name=INTERNE
listener.name.interne.ssl.client.auth=required
ssl.principal.mapping.rules=RULE:^CN=([^,]+).*$/$1/,DEFAULT
sasl.enabled.mechanisms=SCRAM-SHA-512
listener.name.client.scram-sha-512.sasl.jaas.config=org.apache.kafka.common.security.scram.ScramLoginModule required;
authorizer.class.name=org.apache.kafka.metadata.authorizer.StandardAuthorizer
allow.everyone.if.no.acl.found=false
super.users=User:kafka-1;User:kafka-2;User:kafka-3;User:controller-1;User:controller-2;User:controller-3;User:admin-froid
# Contrôleurs : même authorizer.class.name et même super.users, listener.name.controller.ssl.client.auth=required
# Exemples d'autorisations, par un client administrateur authentifié
bin/kafka-acls.sh --bootstrap-server kafka-1:9094 --command-config admin.properties --add \
    --allow-principal User:passerelle-capteurs --operation Write --operation Describe --topic mesures-brutes
bin/kafka-acls.sh --bootstrap-server kafka-1:9094 --command-config admin.properties --add \
    --allow-principal User:flink-froid --operation Write --operation Describe \
    --transactional-id flink-froid- --resource-pattern-type prefixed
```

Sans `super.users` pour les brokers et les contrôleurs, le refus par défaut bloquerait la réplication et le dialogue avec le quorum.

4.4 Plan de remise en service

Arrêt et relance planifiés (phase L), la preuve demandée par le critère 8 :

```
[hôte] bash scripts/tableau-de-bord.sh                        # avant : aucun CRITIQUE (sujet p. 18 : quorum, brokers, ISR…)
bin/kafka-get-offsets.sh --bootstrap-server kafka-1:19092 --topic mesures-brutes   # noter les fins de partition
[hôte] docker compose -f docker-compose-froid.yml stop -t 120  # 120 s pour un arrêt propre (défaut : 10 s)
[hôte] docker compose -f docker-compose-froid.yml start        # contrôleurs d'abord (healthcheck), brokers ensuite
[hôte] for c in froid-controller-1 froid-controller-2 froid-controller-3 froid-kafka-1 froid-kafka-2 froid-kafka-3; do docker
logs $c --since 10m 2>&1 | grep "already formatted"; done
bin/kafka-get-offsets.sh --bootstrap-server kafka-1:19092 --topic mesures-brutes   # mêmes fins qu'avant l'arrêt
grep -h "clean shutdown file" /opt/kafka/logs/server.log    # attendu : « Skipping recovery of … logs … since clean shutdown
file was found »
```

Situation	Procédure	Preuve
Arrêt complet planifié (phase L)	Bloc ci-dessus. Aucun reformatage : chaque nœud affiche « All of the log directories are already formatted. ».	réf. 22/10 : `L2`, `L3` (« already formatted » sur les 6 nœuds, offsets identiques). Poste 25/10 (`J-arret-relance.txt`) : arrêt en 17 s, 6 services `healthy` 145 s après le `start`, « already formatted » sur les 6 nœuds, offsets identiques ; mais brokers tués au bout de 10 s (voir ci-dessous). Avec `stop -t 120` (`J-arret-propre-stop-t120.txt`) : arrêt propre, aucune vérification des journaux au redémarrage.

Situation	Procédure	Preuve
Arrêt brutal (crash de l'hôte ou de Docker)	Même relance. Sans marque d'arrêt propre, Kafka revérifie les segments dont l'écriture sur disque n'est pas garantie : contrôle des CRC, reconstruction des index, troncature d'un lot incomplet. Le signe d'un arrêt brutal est « Recovering N logs … since no clean shutdown file was found » ; la ligne « Recovering unflushed segment 0 … for __cluster_metadata-0 » apparaît, elle, à chaque démarrage.	poste 25/10 (`J-relance-apres-crash.txt`) : crash du moteur de Docker Desktop, 6 nœuds `healthy` 86 s après le `start` , « already formatted » sur les contrôleurs, quorum sain (leader 2, Lag 0).
Panne d'un broker (phase G)	`docker start` , attendre que la sous-réplication revienne à 0, puis rétablir les leaders préférés.	réf. 22/10 (G4, relance des deux brokers arrêtés en G2 et G3) : 65 → 0 partitions sous-répliquées en 12 s, leaders préférés à T+293 s. Poste 25/10 (un broker, §4.2) : 65 → 0 en 75 s.
Panne disque d'un broker	Arrêter le broker et remplacer le disque. Formater le répertoire vide avec le **même** `CLUSTER_ID` et le **même** `node.id` (`kafka-storage.sh format --cluster-id <CLUSTER_ID> --no-initial-controllers --config broker-N.properties` ; au laboratoire, le service le fait seul au démarrage). Relancer : le broker reçoit un nouvel UUID de répertoire et recopie ses répliques depuis les leaders (avec un quota si nécessaire). Attendre 0 partition sous-répliquée, puis lancer l'élection.	poste 25/10 (`J-panne-disque.txt`) : volume du broker 6 supprimé, puis « Formatting metadata directory » au démarrage, avec le même `cluster.id` et un nouvel UUID de répertoire (`iWL-pF9…` → `IE2Qk-…`) ; 0 partition sous-répliquée 83 s après ; les 3 mesures de CAP-001 relues sur le disque neuf.
Panne disque d'un contrôleur	Ne pas laisser le service reformater le volume vide : sa commande (`--initial-controllers`) redonnerait l'ancien UUID de `.env` à un journal vide, qui voterait de nouveau. (1) Retirer l'ancien votant depuis un nœud sain : `kafka-metadata-quorum.sh --bootstrap-controller controller-1:9093 remove-controller --controller-id 3 --controller-directory-id <ancien UUID>` ; les 2 votants restants deviennent indispensables. (2) Formater le disque neuf à la main : `kafka-storage.sh format --cluster-id <CLUSTER_ID> --no-initial-controllers --config controller-3.properties` (nouvel UUID). (3) Démarrer le nœud : il rejoint le quorum comme observateur et rattrape le journal. (4) L'ajouter comme votant : `kafka-metadata-quorum.sh --command-config controller-3.properties --bootstrap-controller controller-1:9093 add-controller` .	procédure, non jouée au TP

Leçon de l'arrêt du 25/10. `docker stop` accorde 10 s avant d'envoyer SIGKILL. Sur le poste chargé, les brokers ont terminé leur arrêt contrôlé en 2,5 s, mais pas la suite de leur arrêt dans ce délai : code 137, puis au redémarrage « *Recovering 65 logs from /var/lib/kafka/data since no clean shutdown file was found* ». Il n'y a eu aucune perte ici, mais sur des centaines de Go cette vérification peut durer des heures. D'où le `stop -t 120` du bloc ci-dessus, ou `stop_grace_period: 2m` dans l'ancre `x-noeud` du Compose (proposition non appliquée au dépôt) ; en production, `TimeoutStopSec` de systemd. Vérifié le 25/10 (`preuves/J-arret-propre-stop-t120.txt`) : avec `stop -t 120` , arrêt en 27 s et codes 143. Les 3 brokers écrivent « shut down completed » 12 à 14 s après SIGTERM, ce qui explique l'échec des 10 s par défaut. Au redémarrage : « *Skipping recovery of 65 logs from /var/lib/kafka/data since clean shutdown file was found* », « already formatted » sur les 6 nœuds et offsets identiques.

Critères de retour en service (go / no-go). (1) Quorum : un leader, 3 contrôleurs en service, retard des votants faible. (2) Brokers 4, 5 et 6 enregistrés, `isFenced: false` . (3) 0 partition sous-répliquée, 0 sous le min ISR, 0 hors ligne. (4) Leaders préférés rétablis. (5) Offsets de fin identiques à ceux d'avant l'arrêt, et groupes capables de reprendre (LAG en baisse). (6) Aucune nouvelle erreur inexpliquée dans les journaux, aucun réglage de démonstration restant. (7) Intervention consignée dans `Command.md` . Les points 1 à 4 et 6 se lisent d'un coup : *[hôte]* `bash scripts/tableau-de-bord.sh` doit afficher « tout est OK » (code retour 0) ; le point 5 se vérifie avec `kafka-get-offsets.sh` et `kafka-consumer-groups.sh` .

4.5 Ce que la réplication ne remplace pas

- **Les sauvegardes.** La réplication copie aussi les erreurs : un topic supprimé, une rétention mal réglée ou des données corrompues par une application se retrouvent aussitôt sur les 3 brokers. Elle ne protège pas non plus d'un sinistre corrélé : site entier, rançongiciel, compte d'administration compromis, bogue touchant tous les brokers.
- **Le `fsync` , dans tous les scénarios.** Par défaut (relevé le 25/10 : `log.flush.interval.messages` = $2^{63} - 1$, `log.flush.interval.ms` non défini), Kafka acquitte un message dès qu'il est dans le **cache mémoire** des répliques de l'ISR, sans attendre l'écriture physique. C'est le choix recommandé par la documentation de Kafka : contre la perte d'un nœud, la réplique perdue se reconstruit depuis les autres, et la réplication tient lieu de `fsync` . Elle ne couvre pas une coupure simultanée de toutes les répliques de l'ISR (coupure électrique des trois zones, par exemple) : des messages confirmés encore en cache seraient perdus. Parades : des zones réellement indépendantes (alimentation, onduleurs), ou `flush.messages` / `flush.ms` sur un topic critique, au prix du débit.
- **L'archivage légal.** Sept jours de rétention ne sont pas un archivage. Les données sont effacées automatiquement, par segments entiers et sans retour (H3). Un topic se modifie par simple commande d'administration (rétention, suppression, compaction) : il n'offre ni stockage immuable (WORM), ni horodatage certifié, ni chaîne de preuve. Kafka n'a pas non plus d'index métier pour retrouver « toutes les mesures de la chambre 12 en mars ». Un archivage suppose une politique de

conservation, d'accès et d'audit, donc un export (Kafka Connect vers un stockage objet verrouillé, ou une base d'archives) : Kafka reste le transport.

5. Présenter les critères 7 et 8 à l'oral (≈ 3 min sur les 10)

Temps	Montrer	Dire
0:00	le `kafka-dump-log.sh` de H1	« Un segment, des lots : `count: 3` ; `producerId` et `sequence` servent à l'idempotence. »
0:30	le tableau avant / après de H2	« 4.2 et 8.7 ont disparu et aucun offset n'est renuméroté ; 9.0 reste parce que 9.2 est dans le segment actif, que le nettoyeur ne compacte jamais. »
1:00	`--time earliest` 0 → 5 et la ligne « Deleting segment » de H3	« La rétention supprime des segments entiers, jugés sur leur message le plus récent ; rien n'est renuméroté. »
1:30	`--describe` avant et après l'élection (I)	« La réaffectation change la liste, l'élection rend le leadership au premier de la liste ; le plan initial est notre retour arrière. »
2:00	*[hôte]* `bash scripts/tableau-de-bord.sh` (garder un terminal hôte ouvert)	« Chaque indicateur a un seuil et une première action. »
2:30	« already formatted » (L), maintenance et panne disque (§4.4)	« Remise en service sans reformatage, un broker à la fois, avec des critères go / no-go. »

Si la démonstration en direct échoue, montrez les fichiers de `preuves/` correspondants, dans le même ordre.

Questions probables (sujet, p. 19). **Q5** : pas dans tous les scénarios. La réplication tient lieu de `fsync` contre la perte d'un nœud, c'est le réglage recommandé par Kafka ; elle ne couvre ni une coupure simultanée de toutes les répliques, ni les erreurs logiques et les sinistres, qui exigent des sauvegardes (§4.5). **Q8** : la compaction fait perdre les valeurs intermédiaires et garde au moins la dernière valeur par clé, avec son offset d'origine (H2). **Q9** : `--generate` propose sans changer le RF et montre l'état actuel, `--execute` lance la copie, `--verify` contrôle la fin et retire les quotas (I-Q1). **Q10** : brokers et contrôleurs sur des hôtes distincts, dans 3 zones réellement indépendantes ; disques dédiés (≈ 80 Go par broker, doc 02) et heap de plusieurs Go ; TLS, SASL et ACL ; supervision Prometheus et Grafana avec ces seuils, et une astreinte ; sauvegardes et archivage ; procédures testées en préproduction et configuration gérée (Ansible ou Strimzi) ; producteurs avec tampon local et job Flink (doc 03, doc 09 et §4.3).

6. Récapitulatif de recette des critères 7 et 8

Élément	Preuve	Statut
Segments et lots lus (H1)	`preuves/H1-segments.txt`, `H1-segments-25-10.txt`	OK
Compaction, offsets inchangés, tombstone, réglages retirés (H2)	`preuves/H2-compaction-*.txt`, `H2-tombstone-et-retrait.txt`, `H2-complement-segment-actif.txt`	OK
Rétention démontrée : début du journal 0 → 5 (H3)	`preuves/H3-retention.txt`	OK
Réaffectation RF 1 → 3, `--verify`, élection, « already elected » (I)	`preuves/I1-I2-generate.txt`, `I3-I4-reaffectation.txt`, `I5-election-preferee.txt`, `I6-throttle-et-retour-arriere.txt`, `I7-rf1-min-isr-effectif.txt`, `json/`	OK
Tableau de bord et plan d'intervention (J)	§4.1, `scripts/tableau-de-bord.sh`, `preuves/J-tableau-de-bord.txt`	OK
Maintenance progressive d'un broker (J)	§4.2, `preuves/J-maintenance-progressive.txt`	OK
Sécurité argumentée (J)	§4.3	OK
Remise en service : arrêt, crash, panne de broker, panne disque (J, L)	§4.4, `preuves/L2-stop-start.txt`, `L3-controle-apres-relance.txt`, `J-relance-apres-crash.txt`, `J-panne-disque.txt`, `J-arret-relance.txt`, `J-arret-propre-stop-t120.txt`	OK

Avant l'oral : tableau de bord à « tout est OK », ce qui couvre aussi l'absence de réglage de démonstration et de topic `demo-*` ; preuves rangées dans `preuves/` ; commandes consignées dans `Command.md`.